# VC Generation

Well-Formed Programs

# Why Preconditions Exist

The grammar accepts more programs than the encoders should consume.

Well-formedness records the shape expected by each VCGen style:

- CFG shape: entry, exit, paths
- definition discipline: SSA or DSA
- assertion discipline: single final failure query
- join discipline: phi nodes or edge-local dynamic definitions

Each form on the next slides is a **contract with a specific encoder** — watch the "consumed by" notes.

# CFG Vocabulary

A CFG has:

- one node per basic block
- edge `b -> b'` when `b` can transfer to `b'`
- successors from outgoing edges
- predecessors from incoming edges

```
entry:
  c := havoc
  if c goto left else right

left:
  goto join

right:
  goto join
```

# SESE

A program is **single-entry single-exit** when:

- `entry` and `exit` are distinguished and distinct
- execution starts only at `entry`
- normal completed executions end at `exit`
- every block lies on an entry-to-exit path

$$\text{SESE} = \text{no unreachable blocks, no dead regions, one normal exit}$$

Consumed by: every encoder in this series.

# SSA

Static single assignment:

- every register is assigned at most once
- phi assignments are a prefix of the join block
- every phi has one incoming value per predecessor

```
left:
  x_left := 1
  goto join

right:
  x_right := 2
  goto join

join:
  x := phi [left: x_left, right: x_right]
```

Consumed by: SeaHorn-style (deck 03) and SeaBMC-style (deck 05).

# DSA

Dynamic single assignment pushes the merge onto incoming edges.

```
left:
  x_left := 1
  x := x_left
  goto join

right:
  x_right := 2
  x := x_right
  goto join

join:
  goto exit
```

The two assignments to `x` are allowed because they occur in sibling predecessors, immediately before the join.

Consumed by: Boogie-style (deck 04).

# SSA To DSA

Before:

```
join:
  x := phi [p1: v1, p2: v2]
  goto exit
```

After:

```
p1:
  x := v1
  goto join

p2:
  x := v2
  goto join
```

DSA can be read as placing the merge assignment on the incoming CFG edge. The reverse direction collects sibling dynamic assignments into a phi.

# SASA

Single-assume single-assert form puts the query at the exit:

```
exit:
  assume pre
  assert post
  halt
```

- exactly one `assume`, exactly one `assert`, both in `exit`
- `pre` accumulates path assumptions
- `post` summarizes assertion obligations

Consumed by: every encoder — the failure query lives in one place.

# Reducing To SASA

Scattered assumes feed `pre`; scattered asserts conjoin into `post`:

Before — two asserts, one assume:

```
entry:
  x := havoc
  assume x > 0
  c1 := x > 0
  assert c1
  y := x + 1
  c2 := y > 1
  assert c2
  halt
```

After — one query at `exit`:

```
entry:
  x := havoc
  c1 := x > 0
  y := x + 1
  c2 := y > 1
  post := c1 and c2
  goto exit

exit:
  assume x > 0
  assert post
  halt
```

If either assert fails, `post` is false — the failure is preserved. Ordering subtleties are why this is **preprocessing**, done once, before any encoder runs.

# Safety Query

For a SASA program, safety is:

$$\forall \xi.\ \text{entry-to-exit}(\xi) \Rightarrow (\text{pre}(\xi) \Rightarrow \text{post}(\xi))$$

The VC asks for the negation:

$$\exists \xi.\ \text{entry-to-exit}(\xi) \wedge \text{pre}(\xi) \wedge \neg\ \text{post}(\xi)$$

# Which Encoder Wants Which Form

| Form | What it fixes | Consumed by |
|---|---|---|
| SESE | CFG shape: no dead regions, one normal exit | all three encoders |
| SSA | one definition per register; merges at phis | SeaHorn (03), SeaBMC (05) |
| DSA | merges as edge-local sibling assignments | Boogie (04) |
| SASA | one final `assume pre; assert post` query | all three encoders |

The conversions (SSA ↔ DSA, fold-to-SASA) are semantic preprocessing — encoder choice starts **after** them.